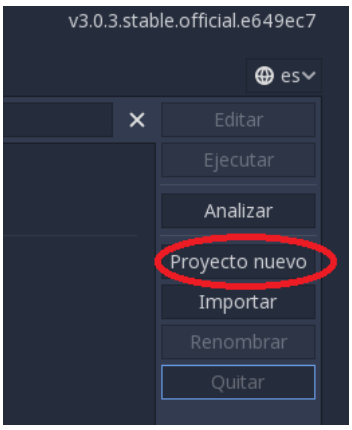
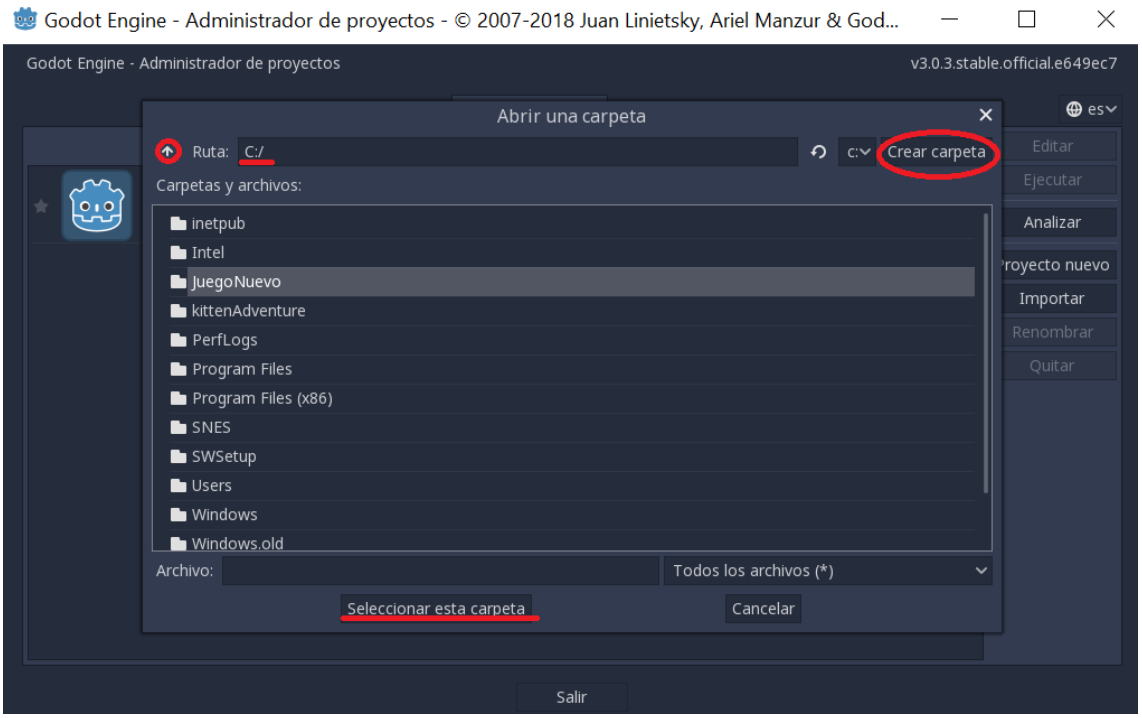
TUTORIAL GODOT – MI PRIMER VIDEOJUEGO

Si ya hemos seguido los pasos de la guía rápida, tendremos a nuestra disposición todo lo necesario para crear un nuevo juego. Para poder empezar ejecutamos Godot y nos aparecerá la pantalla de bienvenida con algunas opciones. En la barra de herramientas de la derecha nos da la opción de crear un *proyecto nuevo*.

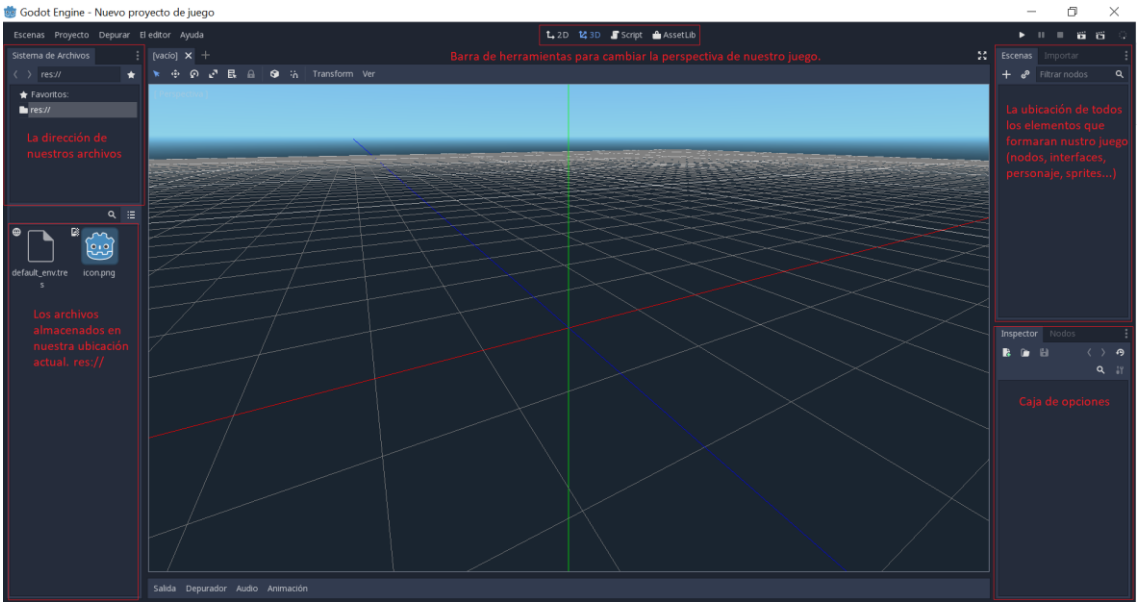


Una vez pinchamos en *proyecto nuevo* debemos establecer la ubicación de la carpeta donde guardaremos todo el contenido de nuestro juego. Entonces pinchamos en examinar y nos aparecerá una nueva ventana donde seleccionaremos **la ruta** donde guardar nuestro proyecto (pinchamos en la flechita hacia arriba para cambiar la dirección de la ruta). En mi caso la guardaré en el disco duro C:

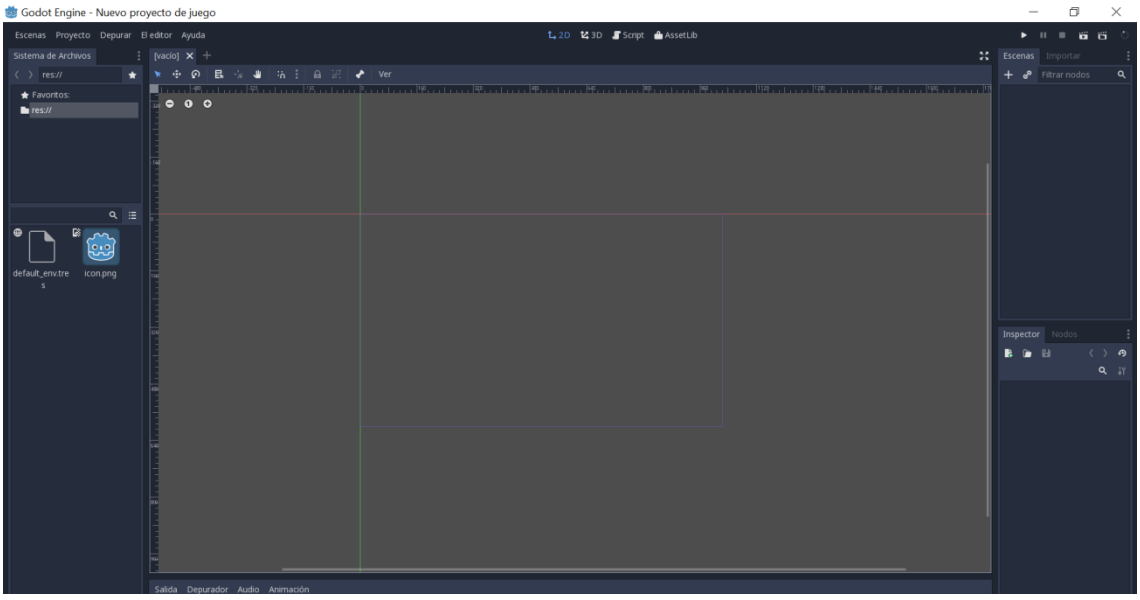


Creamos una nueva carpeta en C:/ a la que podemos llamar como queramos (en nuestro caso la hemos llamado **JuegoNuevo**) y pinchamos en *seleccionar esta carpeta.* Una vez seleccionada pinchamos en crear y editar; creando de esta manera nuestro **nuevo proyecto**.

Ya tenemos nuestro lugar de trabajo generado. Antes de empezar sería conveniente tener un par de cosas claras… En nuestra área de trabajo tenemos cinco zonas bien diferenciadas que os marcamos en la siguiente imagen:



Esta sería nuestra área de trabajo, a la cual tendremos que acostumbrarnos a partir de ahora para poder trabajar. En la barra de herramientas situada en la parte superior de nuestra pantalla podemos cambiar la perspectiva de nuestro juego. Ahora mismo está seleccionada la vista en 3D, pero nosotros vamos a desarrollar un juego en 2D así que seleccionaremos esa pestaña.

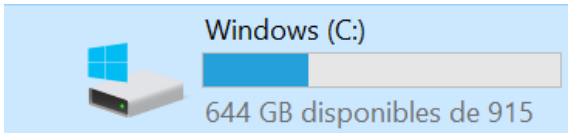


Ahora nuestra pantalla debería aparecer tal que así. El rectángulo que aparece es el área en dos dimensiones donde iremos creando nuestro videojuego. Todo lo que esté colocado fuera de esta área no aparecerá, si ejecutamos nuestro juego, ya que estaría colocado en una región no vista por el jugador. Por tanto todos los elementos que queramos mostrar deben aparecer dentro de este área.

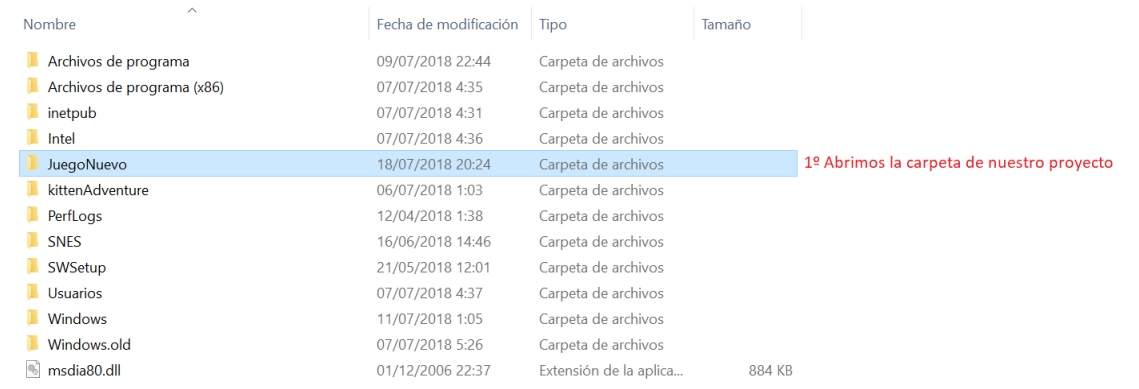
Antes de comenzar vamos a copiar nuestros *sprites* en la misma carpeta que nuestro proyecto. Podéis utilizar los que nosotras hemos creado pero si ya tenéis vuestros propios dibujos o queréis hacer algunos nuevos, podéis hacerlo sin problema. En nuestro caso guardamos el gatito, junto a nuestro fondo verde y los obstáculos (un arbusto, una roca y un charco). Minimizáis **godot** y buscáis **Este Equipo** en vuestro ordenador**:**

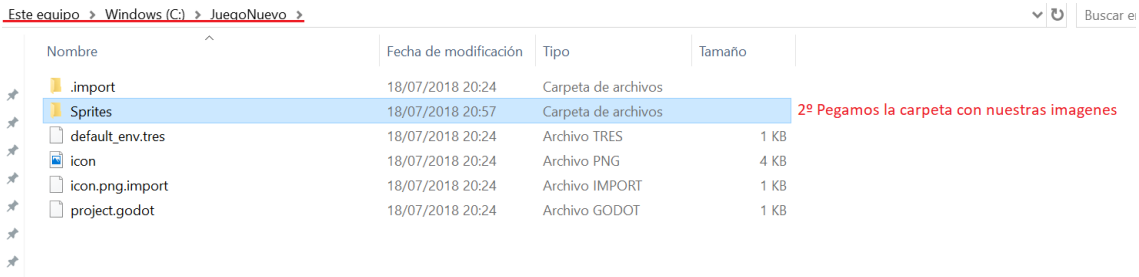


A continuación pincháis en vuestro disco duro (C:):

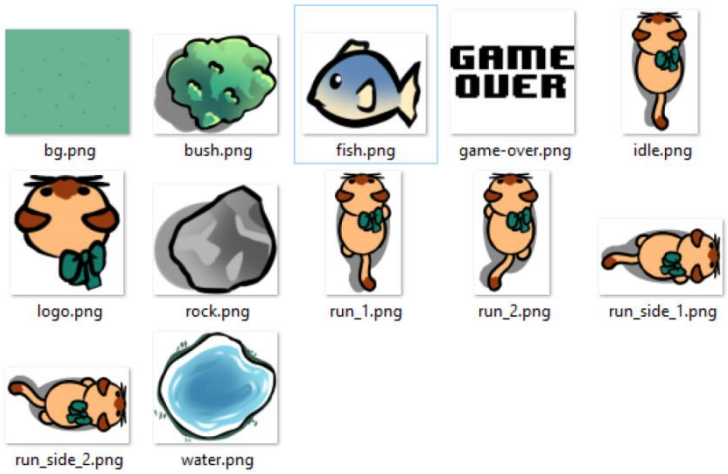


Y tendréis todas las carpetas almacenadas en vuestro disco duro. Debéis ir a la que acabáis de crear para vuestro juego y guardar allí vuestros *sprites*:





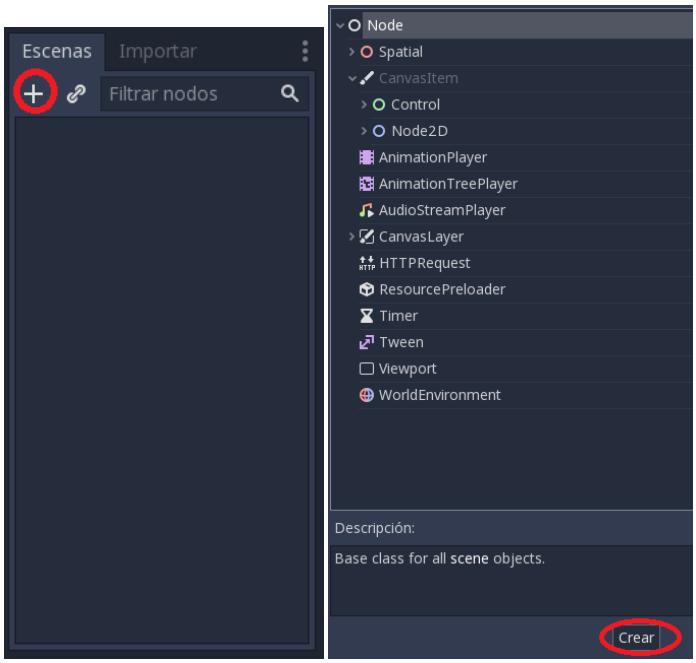
Este sería el contenido de nuestra carpeta *Sprites:*



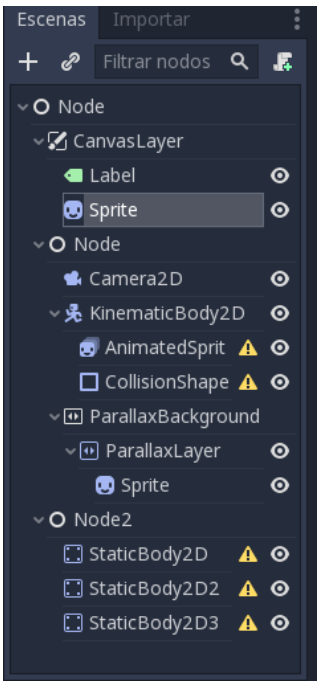
Ahora sí podemos comenzar a trabajar en nuestro videojuego. Nos ubicamos en la caja de escenas (colocada a la derecha de nuestra pantalla) para así poder crear la estructura básica de nuestro videojuego, que será la siguiente:

Node: garden
- i. **CanvasLayer**: UI
  1. **Label**: lives
  2. **Sprite**: fish
- ii. **Node**: map
  1. **Camera2D**: camera
  2. **KinematicBody2D**: kitten
     - a. **AnimatedSprite**: kitten
     - b. **CollissionShape2D**: collision
  3. **ParallaxBackground**: BG
     - a. **ParallaxLayer**: background
       - i. **Sprite**: image
- iii. **Node**: obstacles
  1. **StaticBody2D**: puddle
  2. **StaticBody2D**: bush
  3. **StaticBody2D**: rock

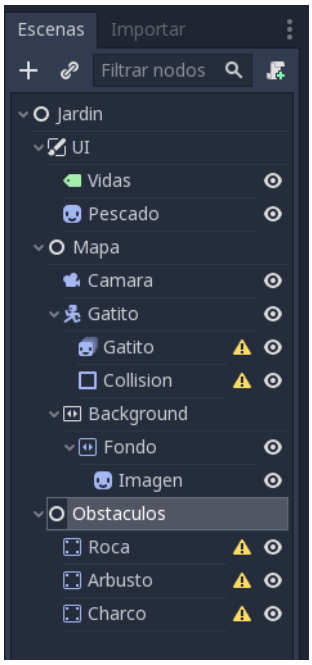
Para crear cada elemento nos vamos a la **caja *escena*** y seleccionamos el +.
Colocamos el primer elemento buscamos ***Node*** y lo creamos:



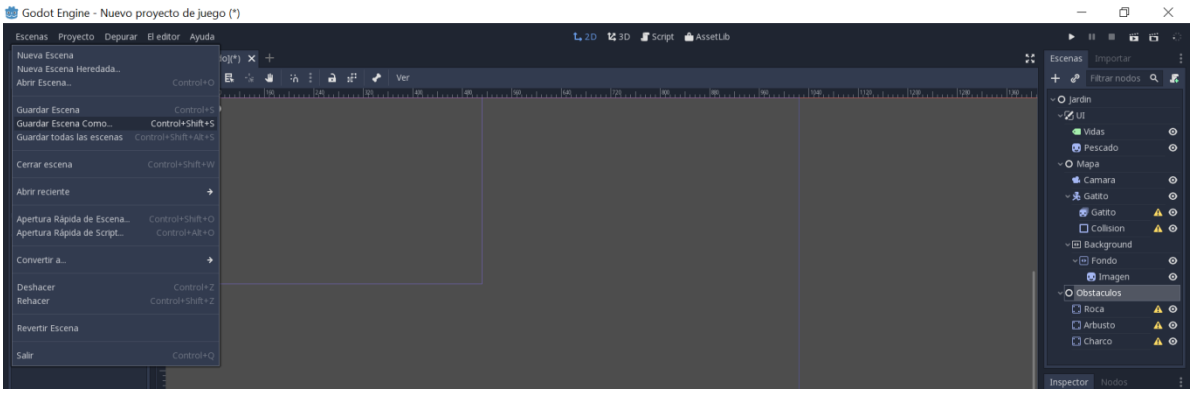
Vamos buscando cada elemento y lo vamos añadiendo tal y como aparece en la estructura que he colocado un poco más arriba (es importante tener cuidado con padres e hijos). Una vez añadido cada elemento debería de quedar algo así:



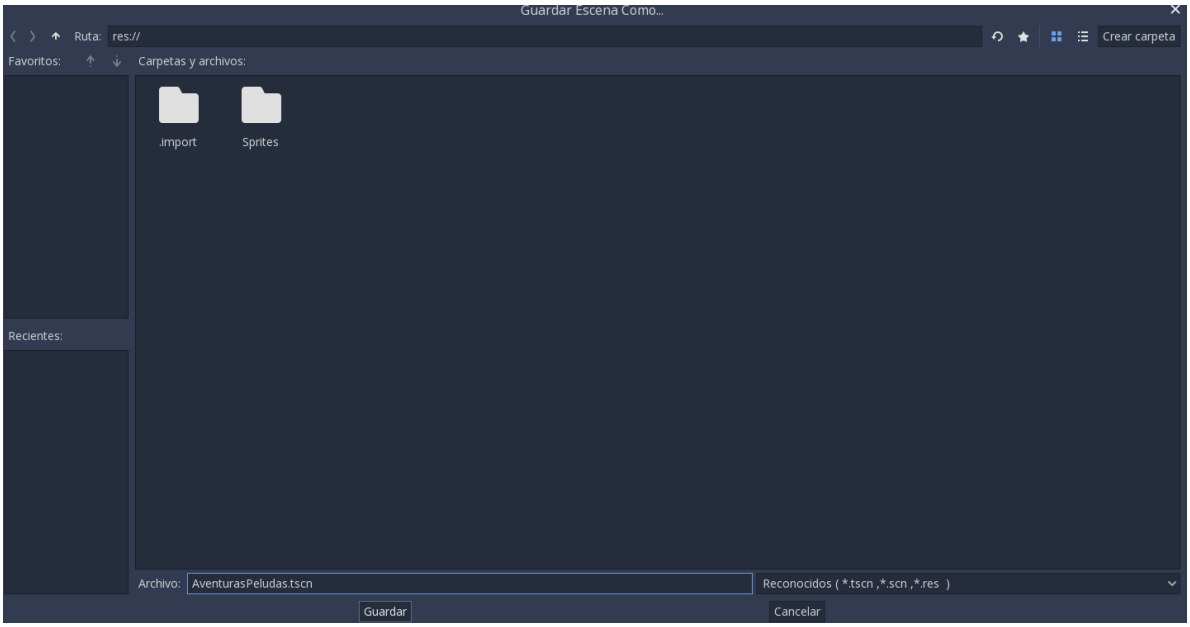
Ya creada la estructura, tal y como se muestra en esta imagen, podemos cambiar el nombre de cada elemento para que entendamos mejor qué es cada elemento. Podéis poner los nombres que queráis, aunque os aconsejamos que utilicéis los mismos que nosotras para no tener problemas. En nuestro caso los hemos llamado así:



Ahora sería conveniente guardar nuestro proyecto, para no perder los cambios (es importante guardar con regularidad, para no perder parte del trabajo que llevemos hecho). Podemos hacerlo desde la barra de tareas (situada en la esquina superior a la izquierda de la pantalla) en la pestaña **Escenas → Guardar Escena Como…**

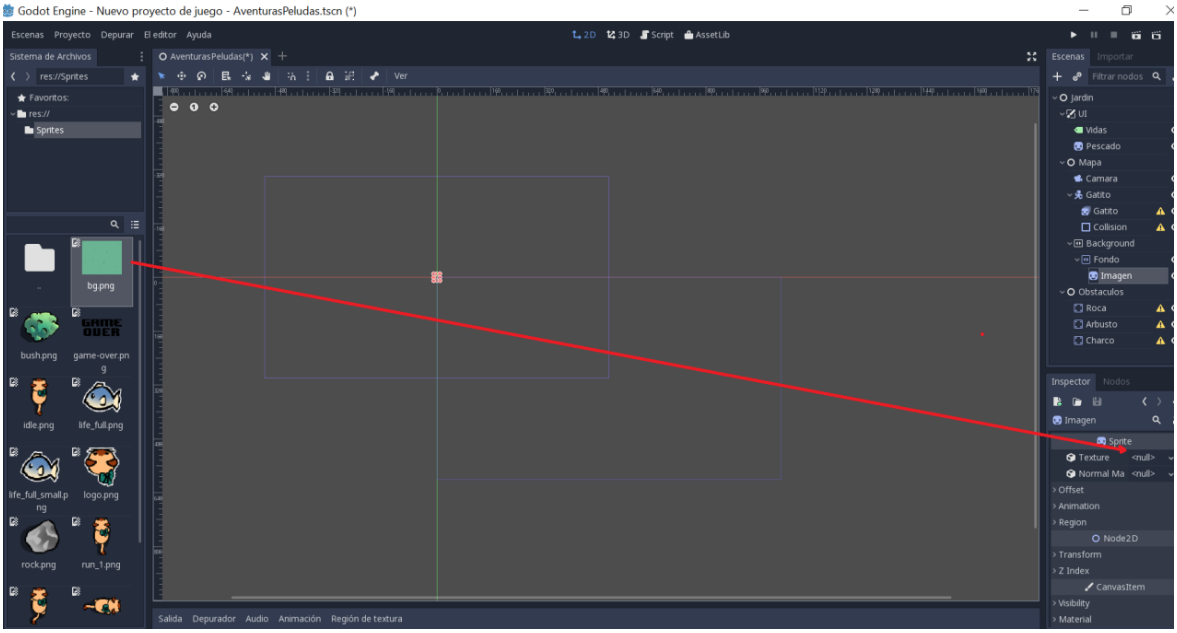


Y guardamos el proyecto pulsando guardar (podéis cambiar el nombre, nosotras lo hemos llamado **AventurasPeludas.tscn**). No borréis el **.tscn** del final si cambiáis el nombre:

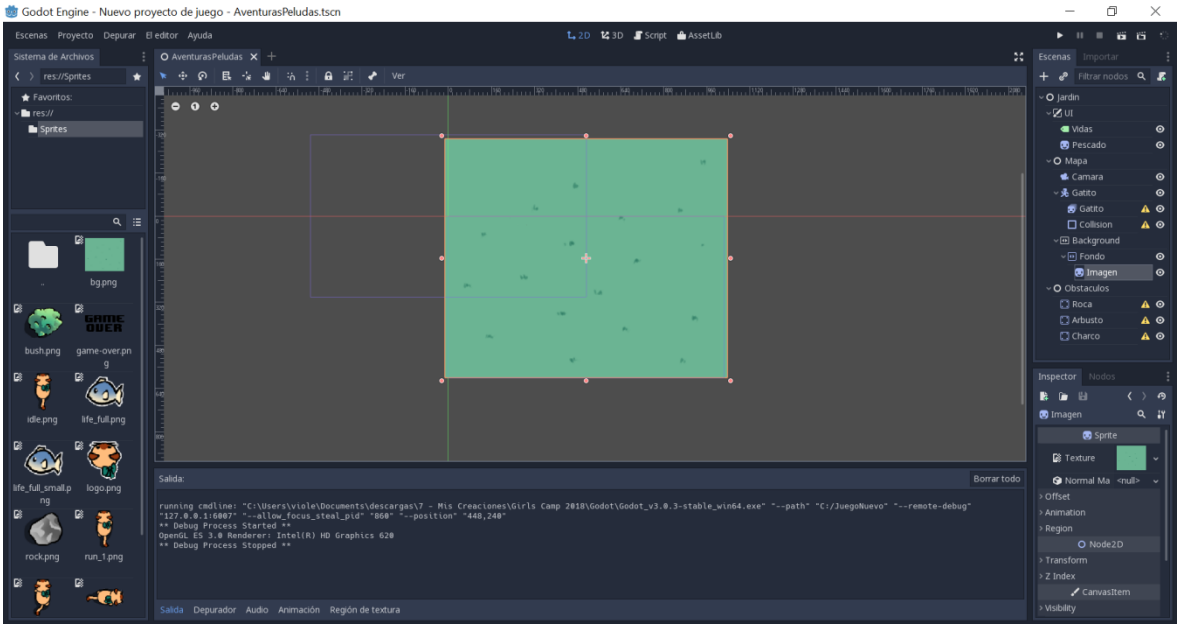


CREANDO UN FONDO PARA NUESTRO JUEGO

Una vez guardado podemos continuar. A continuación vamos a crear el escenario de nuestro juego. La idea es tener un fondo que se mueva de manera infinita hacia abajo. Por tanto seleccionamos el **sprite Imagen** y, a continuación, arrastramos la imagen del fondo (en la carpeta *sprites)* hasta la opción *textura*. Aquí una imagen donde se muestra el proceso:

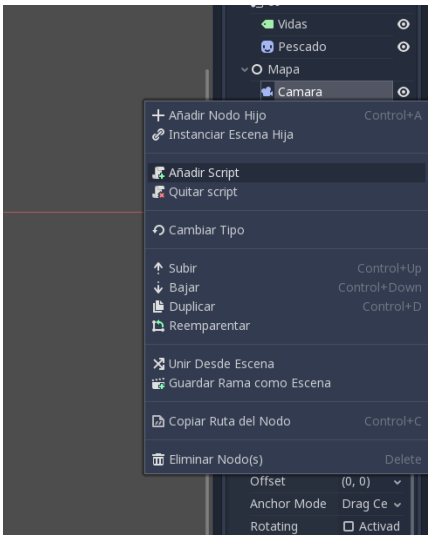


Una vez lo hemos arrastrado hasta la opción texturas, nos debería aparecer el fondo en la pantalla. Podemos seleccionarlo y arrastrarlo para colocarlo encima del rectángulo, que forma el área de nuestro juego, como se muestra a continuación:



Si ejecutamos nuestro juego (seleccionando el “play” colocado en la esquina superior derecha ) nuestro juego se ejecutará y nos mostrará el fondo verde sin movimiento.

Vamos a comenzar a tocar un poquito de código (aunque no debemos de asustarnos). Para ello nos desplazamos al elemento Camara y con el botón derecho del ratón seleccionamos Añadir Script.



Una vez añadido el script, nos aparecerá una nueva pantalla donde podremos añadir las siguientes líneas de código:

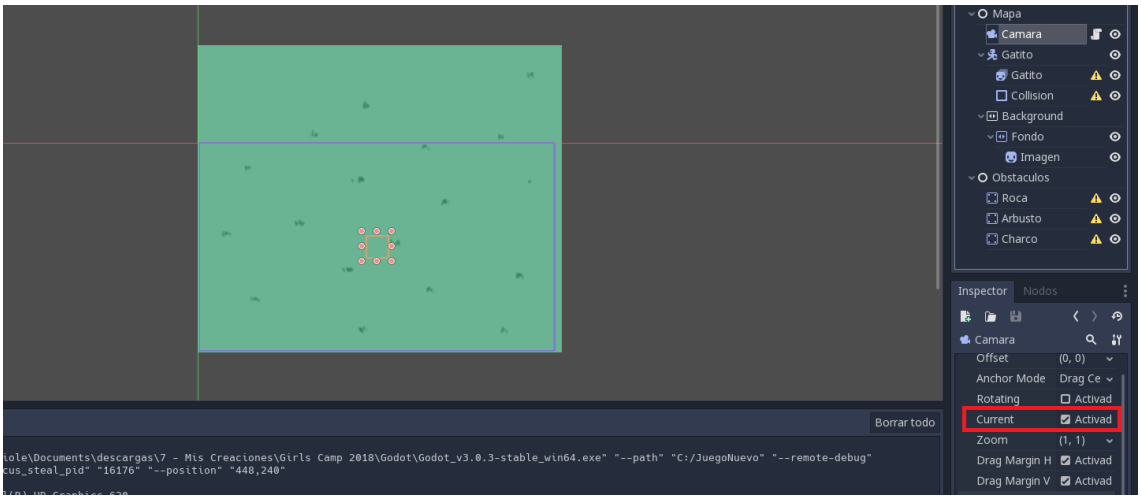
```
extends Camera2D

func _ready():
    set_process(true)

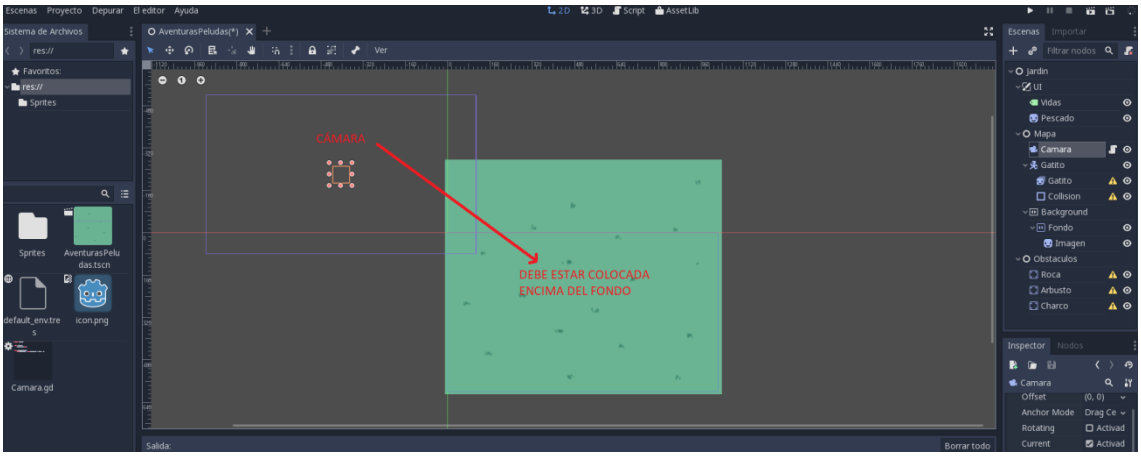
func _process(delta):
    self.position.y = self.position.y - delta * 60

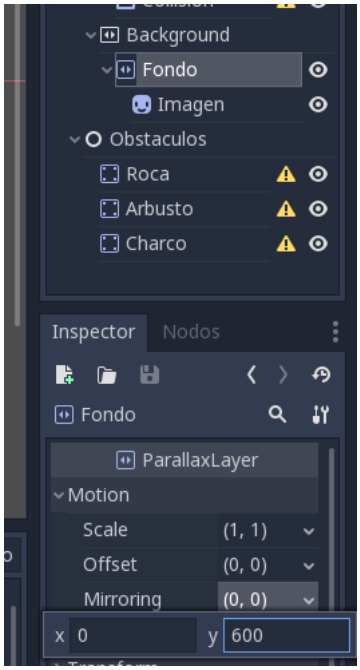
```

En esta línea le estamos modificando la posición a nuestra cámara para que se vaya desplazando por el eje y, logrando mover el fondo como nosotros buscamos. Con esto nuestro fondo debería comenzar, por tanto, a desplazarse. Pero antes de probar si funciona, debemos activar la siguiente opción en las propiedades de la **Camara**:



Es conveniente también comprobar dónde está colocada el área de nuestra cámara. Si el área está fuera de los límites del área principal de nuestro juego, podemos pinchar sobre el rectángulo y desplazarlo para colocarlo en la posición adecuada. Podemos hacer esto manteniendo pulsado **alt** mientras movemos el área con el ratón.

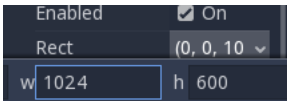




Si os fijáis el fondo no se repite… llega un momento en el que desaparece. Para conseguir que el fondo se repita infinitamente debemos cambiar las propiedades como se muestra en la imagen colocada a la izquierda de estas líneas.

En la opción *Motion* de **Fondo** (ParallaxLayer) establecemos en *Mirroring* que el eje y tenga un valor de 600.

Para conseguir el efecto que deseamos es necesario establecer también el siguiente valor: nos vamos al ***sprite Imagen*** y en sus propiedades seleccionamos *Region* y dentro de esta opción el atributo *Rect*, introduciéndole los siguientes valores:

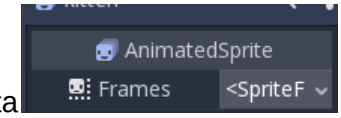


Les otorgamos esos valores, a la altura y anchura, porque es el tamaño general de la mayoría de pantallas. Pero siempre podemos adaptarlo si no se adapta a nuestras necesidades. Es cuestión de ir probando.

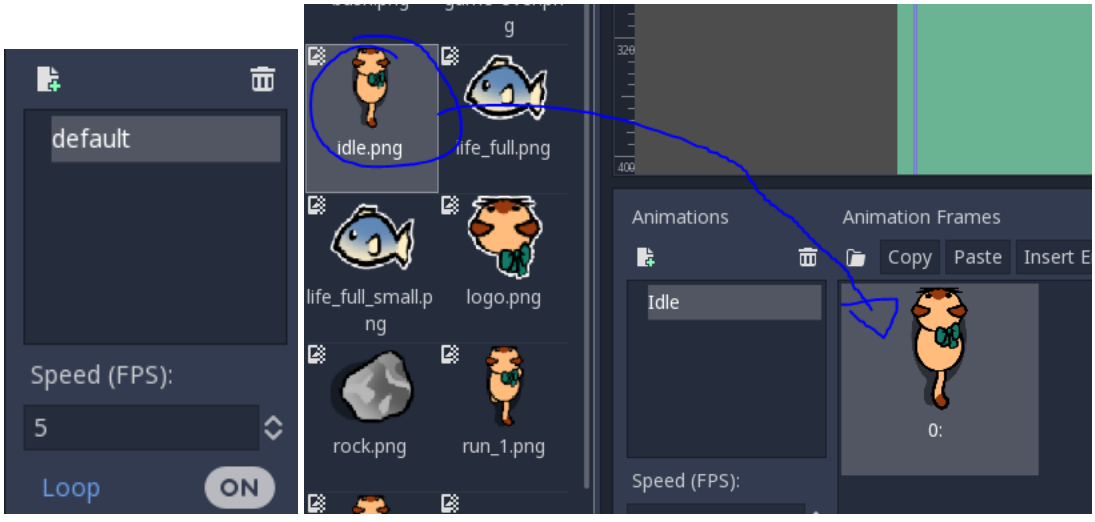
De esta manera estamos consiguiendo que el fondo se repita conforme se va desplazando hacia abajo. Si guardamos los cambios y volvemos a ejecutar el juego (pulsando de nuevo el botón **play** situado en la esquina superior derecha) comprobaremos que se repite indefinidamente. Así conseguimos uno de los principales objetivos de nuestro juego. **¡Ya tenemos el fondo!**

DEMOS MOVIMIENTO A NUESTRO GATITO

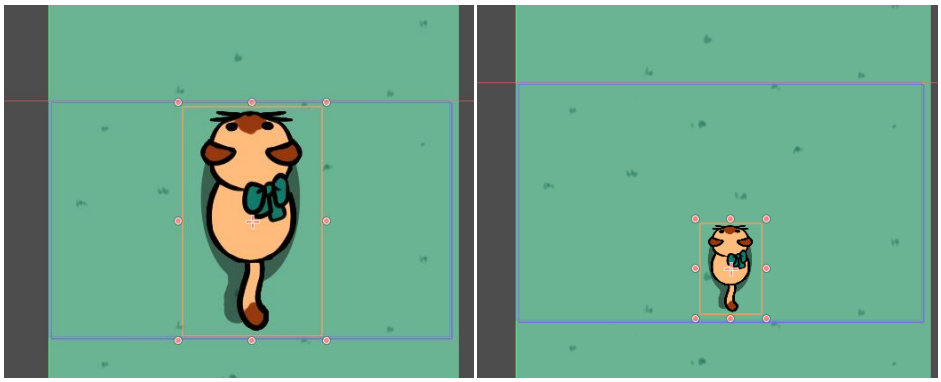
Queremos que nuestro gatito se pueda mover con el teclado. Lo primero que tenemos que hacer es insertar nuestro personaje y así poder tenerlo ubicado. Al principio de este tutorial creamos un *animated sprite,* así que desde aquí crearemos la animación de nuestro gato.

En primer lugar pinchamos en A*nimatedSprite* (gatito) y observamos las opciones. Observaremos la siguiente cajita 

, pinchamos en la pestaña *frames* y seleccionamos **Nuevo SpriteFrames**. Una vez seleccionado, si volvemos a pinchar encima de ***SpriteFrames*** nos aparecerá la siguiente ventana, que nos permitirá crear las animaciones. Primero creamos una animación llamada “Idle” y arrastramos al gatito al cuadrado. Así estaríamos definiendo la posición en la cual el gatito permanece **quieto**.



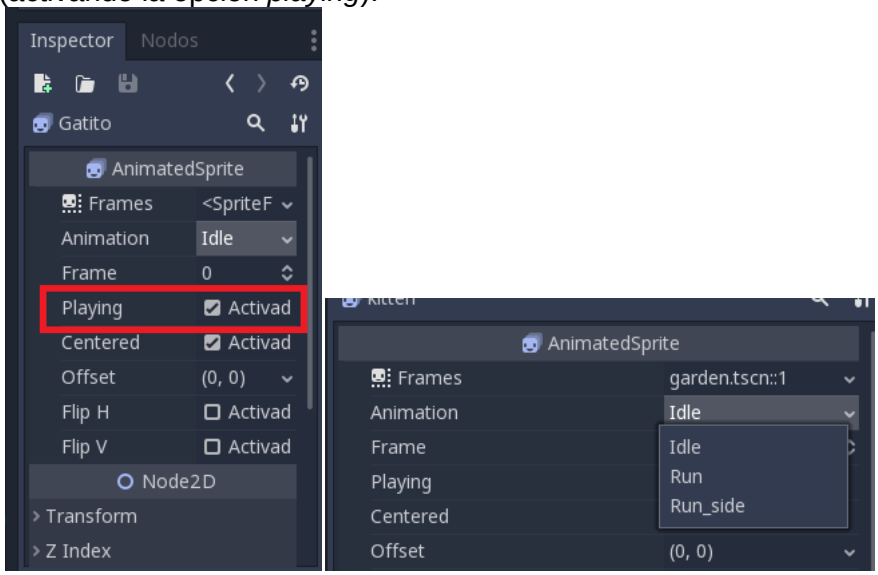
Ahora colocamos bien al gato en el mapa (arrastrándolo, manteniendo pulsado **alt** en nuestro teclado) y una vez lo hemos colocado en su sitio le reducimos el tamaño como se muestra en la siguiente imagen:



Ahora crearemos el resto de animaciones para nuestro personaje. En total serán tres (una quieto, una corriendo de frente y otra desplazándose hacia los lados) *idle, run, run_side.* Para añadir una nueva animación pulsamos el siguiente icono  *:*



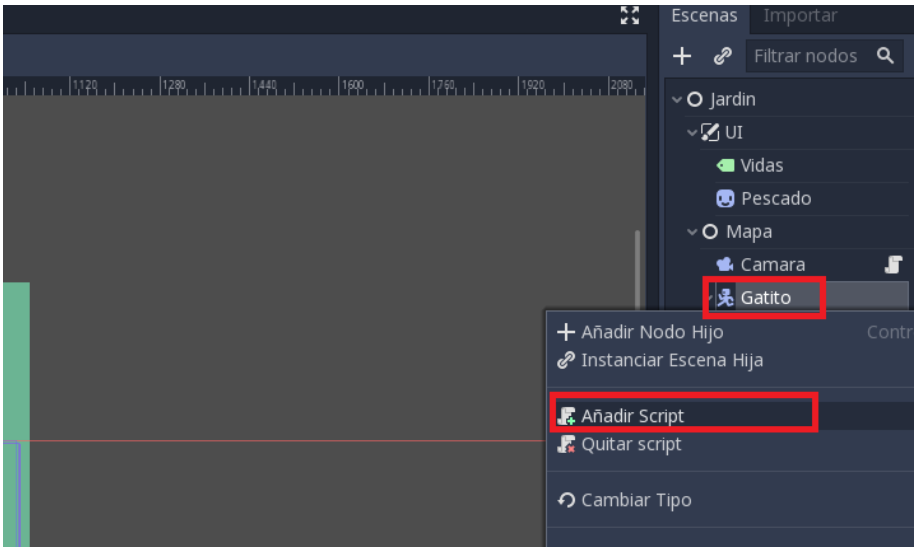
Ponemos por defecto *Idle*, aunque podemos ver la animación cambiando la propiedad del *sprite* (activando la opción *playing*):



Si ejecutamos el juego podemos observar que el gatito no hace nada ni responde a nada. Aquí nos interesan dos cosas:

1. El gatito debe responder a las teclas.
2. El gatito se puede chocar contra obstáculos.

Ahora mismo nos centraremos en que el gatito pueda andar, si pulsamos las teclas de la cruceta de nuestro teclado. Para ello seleccionamos gatito y creamos un nuevo script:



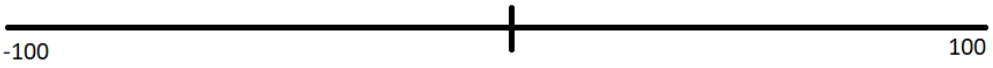
Ahora sí, comenzamos a tocar código de verdad. **¡Hagamos que nuestro gatito se mueva!**

```
1  extends KinematicBody2D
2
3  var motion = Vector2(0,0)
4
5  func _ready():
6      set_process(true)
7      pass
8
9  func _process(delta):
10     if Input.is_action_pressed("ui_right"):
11         motion.x = 100
12     if Input.is_action_pressed("ui_left"):
13         motion.x = -100
14     if Input.is_action_pressed("ui_up"):
15         motion.y = -100
16     if Input.is_action_pressed("ui_down"):
17         motion.y = 100
18
19     move_and_slide(motion)
20     pass
21
```

En primer lugar definimos una variable llamada *motion* (más adelante explicaremos lo que es una variable) con un vector. Como ya deberíamos saber un vector es una magnitud que define una logitud, orientación y dirección, es decir, es como si estuviésemos definiendo una eje (por el cual nos vamos a mover).

Después tenemos dos funciones. Las funciones son las acciones que hará nuestro juego (que el personaje se mueva, definir la posición de la cámara, restablecer un valor… una función puede hacer lo que nosotros queramos). La primera función simplemente establece que el juego se inicie.

En la segunda función estamos declarando que nuestro personaje se mueva al presionar izquierda, derecha, arriba o abajo (esto lo establece una función interna del propio lenguaje de programación, que es is_action_pressed. Las funciones internas son creadas por los desarrolladores e ingenieros que desarrollan un lenguaje de programación, por tanto, nosotros simplemente tenemos que saber darles uso). Y con motion.y = 100, por ejemplo, simplemente le estamos indicando hacia que dirección del eje se debe mover:



Sin embargo, si lo ejecutamos percibimos que se mueve pero… se desplaza hasta el infinito y no tiene animaciones. Por ese motivo vamos a poner unas limitaciones al movimiento. Primero creamos la variable gatito, la cual hace referencia a nuestro personaje:

```
3  var motion = Vector2(0,0)
4  onready var gatito = $Gatito
5
6  func _ready():
7      set_process(true)
8      pass
```

En la variable gatito cuidado con el nombre de la variable *$Gatito*, ya que debe tener el mismo nombre que nuestro *sprite,* es decir, **Gatito** (si vosotras le habéis puesto otro nombre tenéis que comprobarlo 

)

```
func _process(delta):
    if Input.is_action_pressed("ui_right"):
        motion = Vector2(0,0)
        motion.x = 100
        gatito.play("Run")

    elif Input.is_action_pressed("ui_left"):
        motion = Vector2(0,0)
        motion.x = -100
        gatito.play("Run")

    elif Input.is_action_pressed("ui_up"):
        motion = Vector2(0,0)
        motion.y = -100
        gatito.play("Run")

    elif Input.is_action_pressed("ui_down"):
        motion = Vector2(0,0)
        motion.y = 100
        gatito.play("Run")

    else:
        motion.x = 0
        motion.y = 0
        gatito.play("Idle")

    move_and_slide(motion)
    pass
```

Con las siguientes líneas, simplemente estamos restableciendo que el personaje al moverse es como si volviese a estar colocado en la posición 0,0 de nuestro vector. Además de añadirle las animaciones que nosotros hemos dibujado (gatito.play(“Run”)).

Aún así se mueve raro, ¿verdad? Se gira de forma muy brusca. Lo que queremos es que el giro sea más natural, por eso le añadimos las animaciones de giro (aquí lo mismo, **cuidado** con el nombre de las animaciones, *Run_side, Run e Idle*, comprobad que le habéis puesto el nombre correcto).

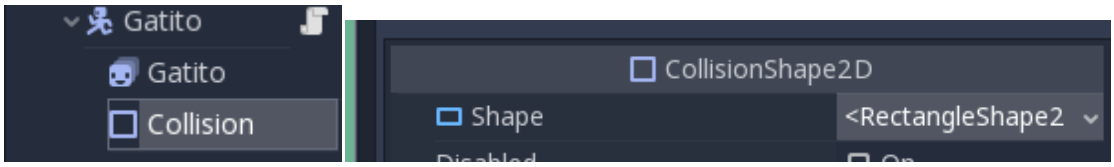
```
if Input.is_action_pressed("ui_right"):
    motion = Vector2(0,0)
    motion.x = 100
    gatito.flip_v = false
    gatito.flip_h = false
    gatito.play("Run_side")
elif Input.is_action_pressed("ui_left"):
    motion = Vector2(0,0)
    motion.x = -100
    gatito.flip_v = false
    gatito.flip_h = true
    gatito.play("Run_side")
```

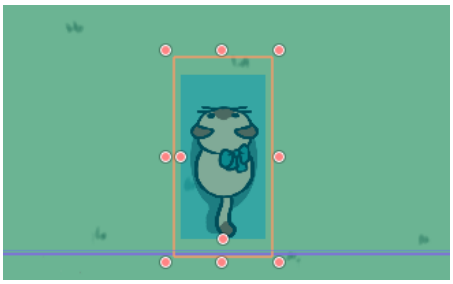
```
elif Input.is_action_pressed("ui_up"):
    motion = Vector2(0,0)
    motion.y = -100
    gatito.flip_v = false
    gatito.flip_h = false
    gatito.play("Run")
```

```
elif Input.is_action_pressed("ui_down"):
    motion = Vector2(0,0)
    motion.y = 100
    gatito.flip_v = true
    gatito.flip_h = false
    gatito.play("Run")
```

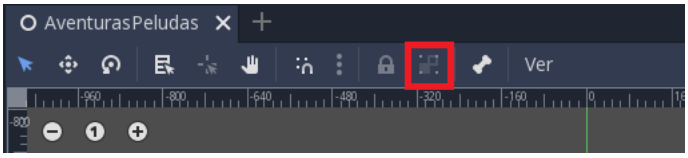
COLISIONES

Vamos a configurar el objeto de colisión. El gatito tenía una capa de colisión, por tanto, vamos a darle el tamaño del gatito. Le estableceremos forma rectangular a la colisión, primero, determinando que la zona azul detectará choques. Si pulsamos en **Collision** y en opciones pinchamos en **shape → RectangleShape2D** nos aparecerá un rectángulo que podemos arrastrar y colocarlo encima de nuestro personaje. Dándole la forma suficiente para que ocupe todo el área de nuestro personaje. Con esto estamos definiendo que nuestro gato es un objeto que se puede chocar.





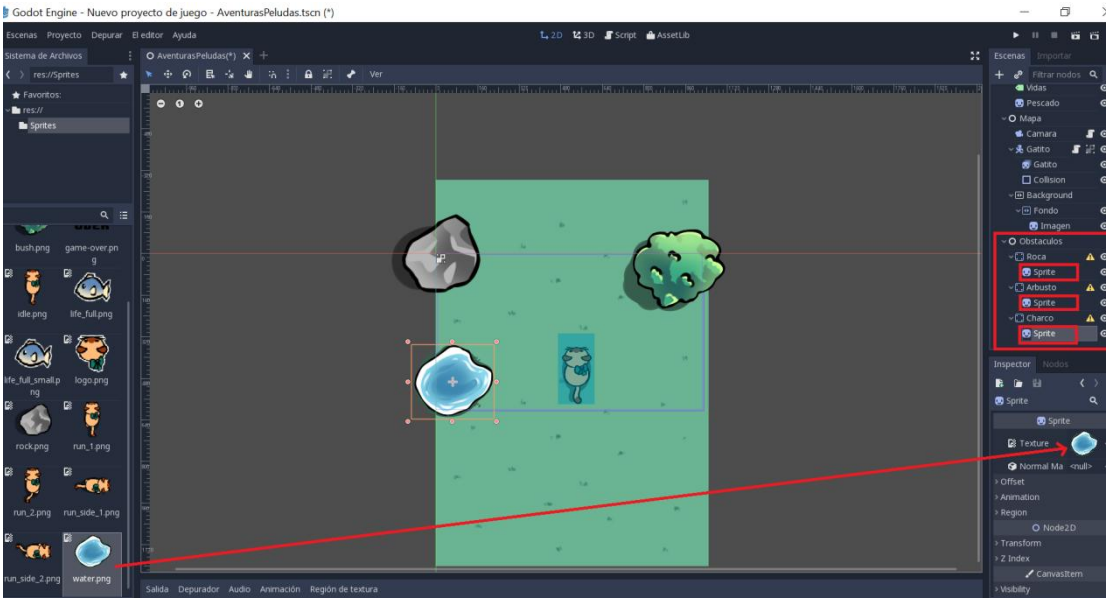
Ahora tenemos que juntar todos los objetos en uno solo. Para poder hacerlo seleccionamos el padre (**Gatito**) y pinchamos el siguiente botón (colocado en la barra superior de nuestra área de trabajo).



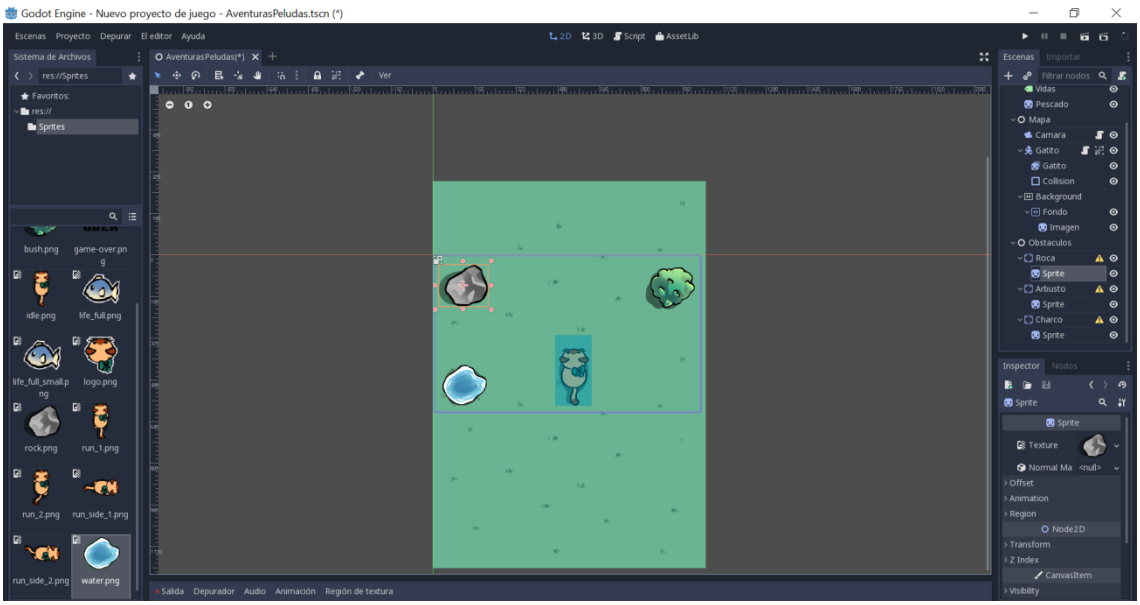


Ahora debemos definir los obstáculos de nuestro juego, que en nuestro caso son una roca, un arbusto y un charco. La idea es que los objetos aparezcan de forma aleatoria por el escenario conforme avanza nuestro personaje. Si el gatito choca con alguno de esos objetos entonces lo ideal es que el gatito pierda una vida. Pero de momento solo nos interesa que detecte la colisión.

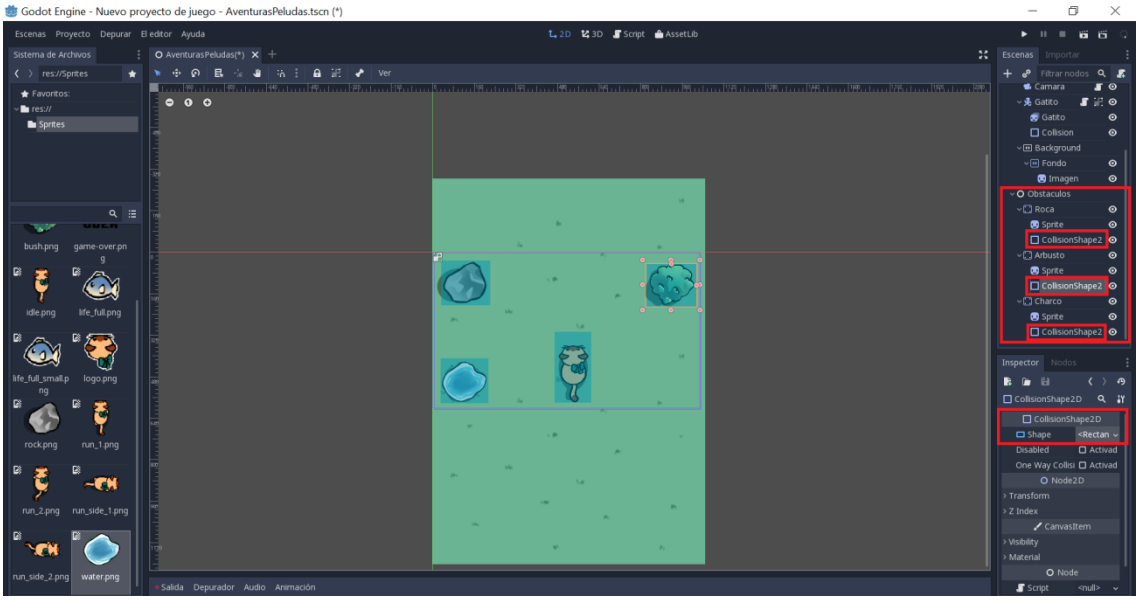
Para ello nos vamos a *Obstaculos* y creamos un *sprite* para cada obstáculo, como se muestra en la siguiente imagen:



Colocamos en cada *sprite* la textura de nuestro objeto, arrastrando la imagen hasta la caja de opciones en *Texture*. Las imágenes se verán tal y como aparecen en la pantalla. Podemos reducirles el tamaño y colocarlas mejor en el área del juego. Para moverlas podemos mantener pulsado **alt** mientras arrastramos la imagen. Al final nos quedaría algo así:



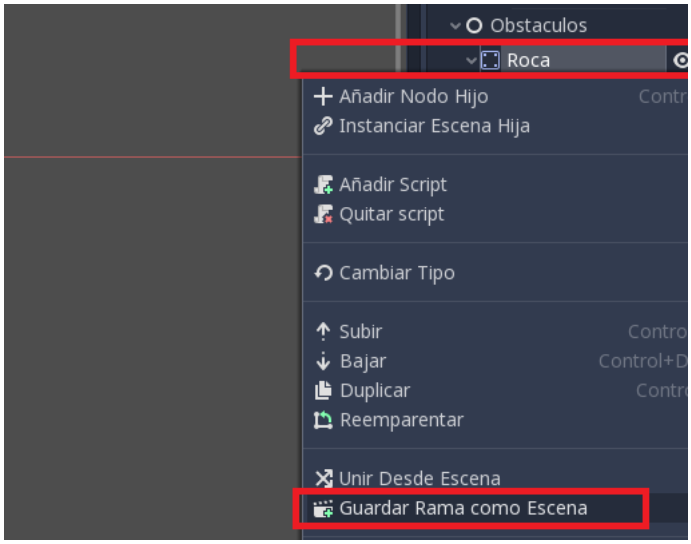
Ahora debemos crear, dentro de cada obstáculo, un *collisionShape*. De esta manera definiríamos que cada objeto tenga un área de colisión. Así pues generamos un *collisionShape* dentro de cada obstáculo como se muestra a continuación:



Como hiciéramos con el gato, anteriormente, definimos en las opciones de cada *collisionshape2D* un *RectangleShape2D* y definimos su área de colisión para que ocupe todo el objeto (como se puede observar en la imagen). Recordad que el área se puede desplazar por la pantalla si mantenemos pulsada la tecla **alt**.

Si ahora ejecutamos nuestro juego nos daremos cuenta que el gato no puede avanzar cuando se choca con cualquiera de los obstáculos, repartidos por el escenario. Ahora nos interesa que esos obstáculos se generen de forma aleatoria por el escenario.

Antes debemos guardar cada obstáculo como una escena independiente, para poder llamarlos una vez estemos programando (tenemos que hacer referencia a esos objetos creados de alguna forma). Para ello pinchamos en **Arbusto**, por ejemplo, pulsamos el botón derecho del ratón y *guardamos rama como escena*:



Hacemos lo mismo para nuestro arbusto y charco. Una vez guardado, ya podemos volver al código que habíamos generado en gatito. Pinchamos en el script 

y… **¡a programar!**

En el inicio de nuestro *script*, justo debajo de **onready var gatito = $Gatito** generamos las siguientes líneas de código:

```
var objectInstance

#Cargamos los obstaculos
var cameraPosition_y = 0

onready var charco = preload("Charco.tscn")
onready var arbusto = preload("Arbusto.tscn")
onready var roca = preload("Roca.tscn")
```

Aquí estamos generando una serie de variables para guardar ciertos datos. **Una variable es como un recipiente donde almacenar datos.** Pero para poder usar ese recipiente debemos declararlo y eso estamos haciendo aquí: declarando y creando nuestros recipientes para almacenar los datos que necesitamos usar.

A veces puede ser un recipiente para guardar solo números o cadenas de caracteres (letras, frases, etc) o incluso objetos. En la variable **charco**, por ejemplo, estamos guardando el elemento, que hemos definido previamente en este tutorial, como un objeto con el que nuestro gatito podría chocar. Con *cameraPosition_y* definimos la posición inicial de nuestra cámara que será **cero.**

Una vez hemos creado las variables (recipientes) donde guardar los datos que necesitamos, vamos a operar. Para ello vamos a crear una nueva función que se va a llamar *func random_obstaculos(y).* La podemos crear justo al final de nuestro script:

```
    else:
        motion.x = 0
        motion.y = 0
        gatito.play("Idle")

    move_and_slide(motion)
    pass

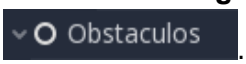
#Generar obstaculos de manera aleatoria
func random_obstaculos(y):

    var x = randi()%800+240;

    match randi() % 3:
        0:
            objectInstance = arbusto.instance()
        1:
            objectInstance = charco.instance()
        2:
            objectInstance = roca.instance()

    objectInstance.position = Vector2(x,y)
    get_parent().get_parent().get_node("Obstaculos").add_child(objectInstance)
```

Con estas líneas estamos consiguiendo que nuestros objetos **arbusto, charco y roca** se vayan generando de forma arbitraria por el escenario conforme el gatito va andando. Aquí hay que tener mucho cuidado con los nombres. **arbusto.instance()**, por ejemplo, está haciendo referencia a la variable que hemos creado unas líneas más arriba y debe llamarse exactamente igual (si no nos saltará un error porque no encuentra esa variable. Es como si vais por la calle y alguien os llama por el nombre de Pepa, cuando vosotras no os llamáis así… Ignorareis a esa persona, ¿verdad? Pues aquí pasa igual).

También hay que tener cuidado con **getNode("Obstaculos")** el "Obstaculos" debe llamarse igual que → 

. **¡MUCHO CUIDADO CON ESTO! ¡Revisad muy bien los nombres!**

Una vez creada la función *random_obstaculos* nos vamos a la función

func_process(delta) e introducimos un nuevo valor para la posición de nuestra cámara:

```
16
17  func _ready():
18      set_process(true)
19      pass
20
21  func _process(delta):
22
23      cameraPosition_y = get_parent().get_node("Camara").get_camera_position().y
24
25  #Obstaculos
26      if (int(cameraPosition_y) % 200 == 0):
27          # generamos obstáculos
28          random_obstaculos(cameraPosition_y-100)
29
30
31  #Movimiento del gato
32      if Input.is_action_pressed("ui_right"):
33          motion = Vector2(0,0)
34          motion.x = 100
35          gatito.flip_v = false
36          gatito.flip_h = false
```

Aquí estamos llamando a la función que acabamos de crear (random_obstaculos) para poder generar los obstáculos, teniendo en cuenta la posición de la cámara. Es muy importante tener, de nuevo, en cuenta el nombre de **.get_node("Camara")** debe ser igual a → 

porque estamos haciendo referencia a la cámara que nosotros creamos al principio de este tutorial.

Una vez copiado este código correctamente, si ejecutamos nuestro juego comprobaremos que los obstáculos se van generando conforme el gatito va avanzando. **¡Misión cumplida!**